game, we instead store the experiences in a “memory” store and then sample mini-batches from the memory for training.

We will build an experience replay class that contains a list to store tuples of experiences, where each tuple is of the form (S_t,a_t,r_t,S_{t+1}). The class will also have methods to add a memory and sample a mini-batch.

Listing 8.5. Experience replay

```
from random import shuffle
import torch
from torch import nn
from torch import optim
import torch.nn.functional as F

class ExperienceReplay:
    def __init__(self, N=500, batch_size=100):
        self.N = N                                          1
        self.batch_size = batch_size                        2
        self.memory = []
        self.counter = 0

    def add_memory(self, state1, action, reward, state2):
        self.counter +=1
        if self.counter % 500 == 0:                         3
            self.shuffle_memory()

        if len(self.memory) < self.N:                       4
            self.memory.append( (state1, action, reward, state2) )
        else:
            rand_index = np.random.randint(0,self.N-1)
            self.memory[rand_index] = (state1, action, reward,
state2)

    def shuffle_memory(self):                               5
        shuffle(self.memory)

    def get_batch(self):                                    6
        if len(self.memory) < self.batch_size:
            batch_size = len(self.memory)
        else:
            batch_size = self.batch_size
        if len(self.memory) < 1:
            print("Error: No data in memory.")
            return None
                                                            7
        ind = np.random.choice(np.arange(len(self.memory)), \
   batch_size,replace=False)
```